

Hardware-Software Co-design for Distributed Quantum Computing

Ji Liu^{*§}, Allen Zang^{†§}, Martin Suchara[‡], Tian Zhong[†], and Paul D Hovland^{*}

^{*}Argonne National Laboratory [†]The University of Chicago [‡]Microsoft Azure Quantum [§]Equal contribution
{ji.liu, hovland}@anl.gov, {yzang, tzh}@uchicago.edu, msuchara@microsoft.com

Abstract—Distributed quantum computing (DQC) offers a pathway for scaling up quantum computing architectures beyond the confines of a single chip. Entanglement is a crucial resource for implementing non-local operations in DQC, and it is required to allow teleportation of quantum states and gates. Remote entanglement generation in practical systems is probabilistic, has longer duration than that of local operations, and is nondeterministic. Therefore, optimizing the performance of probabilistic remote entanglement generation is critically important for the performance of DQC architectures. In this paper we propose and study a new DQC architecture that combines (1) buffering of successfully generated entanglement, (2) asynchronously attempted entanglement generation, and (3) adaptive scheduling of remote gates based on the entanglement generation pattern. We show that our hardware-software co-design improves both the runtime and the output fidelity under a realistic model of DQC.

I. INTRODUCTION

Quantum computing is able to efficiently solve important classes of computational problems that are hard to solve using classical computers [1]. To make the computational power of quantum computers beneficial for real-world applications, quantum computing platforms need to be scaled beyond the size of state-of-the-art systems with limited number of qubits that have been demonstrated so far. However, placement of a large number of qubits within one monolithic quantum processing unit is expected to result in system performance degradation due to cross-talk between nearby physical qubits, limited capability of storing the physical qubits on a single chip with a small footprint, and increased complexity of controlling the individual qubits. Scaling up monolithic QPUs will be therefore constrained by various technical obstacles that are difficult to overcome.

In contrast to monolithic QPUs, distributed quantum computing (DQC) [2]–[4] is a paradigm which aims to interconnect multiple monolithic QPUs together with the goal to allow evaluation of quantum circuits whose sizes exceed the number of qubits in a single QPU. DQC is viewed as an important milestone in scaling quantum computers, as described in the roadmaps of many quantum hardware manufacturers [5], [6]. DQC allows finding the sweet spot for the size of the individual QPUs, and therefore mitigates the aforementioned monolithic QPU scaling difficulties. At the same time, DQC introduces new challenges. Entanglement between the QPUs is the most important resource in DQC, which is consumed to implement remote multi-qubit gates. The generation of remote entanglement involves photon transmission, photonic Bell state measurement for heralding of success, and feedforward of measurement results. These processes result in a probabilistic success of remote entanglement generation and a longer cycle time for each attempt compared to local quantum operations. These entanglement generation features represent a significant bottleneck in the DQC performance. Implementation of remote gates requires prior successful entanglement generation, leading to potential gate delays that may further cascade through the system due to gate dependencies. When initialized qubits sit idle, this introduces additional decoherence leading to circuit output fidelity degradation as well as slower evaluation of the circuit.

In this paper, we study a new DQC architecture which incorporates hardware-software co-design. The three key principles of our architecture are:

- 1) Use of three different types of hardware qubits: communication qubits which are used for remote entanglement generation, buffer qubits which store the successfully generated entangled states, and data qubits which are used for quantum circuit evaluations.
- 2) Asynchronous attempts to generate remote entanglement. This smoothens the temporal pattern of successfully generated entanglement, leading to better time resource management.
- 3) Division of the quantum circuit into segments and identification of equivalent variants of circuit segments obtained by commuting remote gates to allow more efficient adaptive scheduling of remote gates by using information about the success of entanglement generation.

This paper is organized as follows: In Sec. II we review the necessary background and survey related work. We describe our architectural framework in Sec. III, and the evaluation methodology in Sec. IV. Our evaluation results are presented in Sec. V. Finally, we conclude in Sec. VI.

II. BACKGROUND

In this section, we review basics of quantum information, the mechanism of heralded remote entanglement generation, and the implementation of basic non-local operations in DQC. Then we review relevant literature.

A. Quantum information basics

Quantum information is encoded in continuous complex probability amplitudes. Noiseless (pure) quantum states are linear combinations of eigenstates of the physical systems, i.e. quantum superpositions. Quantum states can be written as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ with $\alpha, \beta \in \mathbb{C}$, where $|0\rangle$ and $|1\rangle$ are orthonormal bases of a two-dimensional Hilbert space, and normalization requires $|\alpha|^2 + |\beta|^2 = 1$. In quantum computing, an entire quantum circuit is a unitary operator U , s.t. $U^\dagger U = U U^\dagger = I$, where the dagger in the superscript denotes composition of complex conjugation and transpose, and I is the identity operator.

Entanglement is another key feature of quantum mechanics, and also one of the most important resources in DQC. A canonical example of entangled states are the 4 orthonormal *Bell states* (*EPR pairs*) of two qubits: $|\Phi^\pm\rangle = \frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$, $|\Psi^\pm\rangle = \frac{1}{\sqrt{2}}(|01\rangle \pm |10\rangle)$. The Bell states can be transformed into each other via applications of single-qubit operations known as *Pauli operators* which are defined as follows: $X|0\rangle = |1\rangle$, $X|1\rangle = |0\rangle$ (also denoted as $\oplus$), $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$, and $Y = iXZ$.

Environment perturbations will cause decoherence by imposing errors on quantum states. Decoherence generally transforms quantum states to mixed states, i.e. a statistical mixture of pure quantum

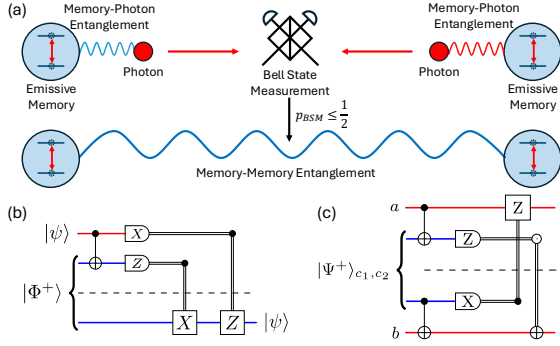


Fig. 1. Preliminaries of DQC. (a) Heralded remote entanglement generation. (b) Quantum state teleportation circuit. (c) CNOT teleportation circuit. The dashed lines denote the partition of two local parties. The blue lines represent the Bell pair resource and the red lines represent data qubits.

states. A common error model is the Pauli channel that applies each of the three Pauli operators and the identity operator with certain probabilities. The *fidelity* of the mixed output quantum state ρ with respect to the ideal pure output state $|\psi\rangle$: $F = \langle\psi|\rho|\psi\rangle$, is a key performance metric.

B. Heralded remote entanglement generation

Heralded remote entanglement generation [7]–[10] can establish entanglement between two local systems that have no direct physical interaction. The effective interaction is realized in the following way:

- 1) The local systems emit photons which are entangled with themselves. Such matter-photon entanglement can be generally expressed as a standard Bell state.
- 2) The two emitted photons are transmitted through a physical channel to an intermediate photonic Bell state measurement (BSM) station.
- 3) The photonic BSM realizes an effective entanglement swapping [11], which conditioned on success projects the quantum state of two remote systems into a Bell state.
- 4) The classical BSM result is communicated with the two end nodes, which heralds whether the remote entanglement generation is successful.

The above processes are visualized in Fig. 1(a).

C. Nonlocal operations with entanglement

In DQC, nonlocal multi-qubit gates between QPUs can be implemented through teleportation. We may either teleport qubits [12] from one QPU to another QPU, perform local multi-qubit gates and teleport the qubit back, or directly teleport the multi-qubit gates [13]. These approaches require Bell pairs as resources that are consumed for each attempt, local operations on each QPU, and potential feedforward Pauli frame correction. The circuit for quantum state teleportation is depicted in Fig. 1(b). Fig. 1(c) shows the specific circuit that teleports a CNOT gate.

D. Related work

Related to this work, AutoComm [14] and QuComm [15] are state-of-the-art approaches which take into account quantum circuit structure to harness the benefits of DQC. However, these works ignore one important feature - the probabilistic nature of EPR pair generation. AutoComm identifies burst communication patterns in the input program and utilizes state or CNOT teleportation to allow remote operations. It compiles the circuit and schedules remote operations

offline, making it unaware of the real-time EPR pair generation pattern. QuComm identifies multi-node collective communication, and uses “buffer” qubits which store generated EPR pairs while communication qubits are freed up to be able to keep generating EPR pairs.

Several research papers have sought to reduce the communication overhead of distributed quantum programs by exploring various qubit partitioning and mapping techniques [16]–[23]. Some work [24]–[26] focuses on tackling the mapping problem in multi-core quantum computers. Additional work incorporates a recent trend and use reinforcement learning for DQC compiling [27], [28]. These approaches are orthogonal to our work. Finally, a hardware-focused body of work recently emerged as well [29]–[32].

III. PROPOSED ARCHITECTURAL FRAMEWORK

To motivate our proposed architecture, we first analyze heralded remote entanglement generation for DQC. Then we explain the three principles of our architectural design.

A. Technical considerations of remote entanglement generation

The runtime of DQC in realistic heralded remote entanglement generation is affected by the success probability and the cycle time per attempt. The final fidelity of the DQC circuit output is affected by the quality of the generated entanglement (fidelity of the generated Bell pair). However, entanglement infidelity only injects errors into the circuit locally when implementing remote gates, similar to other noisy local gates.

Success probability per attempt: The success probability of a heralded remote entanglement generation attempt is determined mainly by the following factors:

- 1) Local photon-qubit entanglement generation probability p_{pq} during one entanglement generation cycle (attempt).
- 2) Photon transmission loss. For typical low-loss optical fiber the efficiency $\eta_t = \exp(-L/L_{att})$ is a function of channel length L where the characteristic attenuation length for optical fiber is $L_{att} \approx 20$ km.
- 3) In practice photon BSMs are typically implemented using linear optical components whose maximal success probability is $1/2$ [33]. In addition, there are other factors such as optical coupling efficiencies and detector efficiencies. We denote the total success probability of the photon BSM as p_s .

Thus, the success probability of one entanglement generation attempt is $p_{succ} = p_{pq,1}p_{pq,2}\eta_{t,1}\eta_{t,2}p_s \leq 1/2$. Note that both sides must succeed with the transmission.

Cycle time per attempt: The following three processes are the main contributors to the cycle time:

- 1) For realistic DQC, the allowed waiting time for photon emission should be a fixed value (cutoff time) otherwise the probabilistic nature of emission might lead to an excessively long wait for the emission. On the other hand, the chosen cutoff time will affect the photon emission probability for every attempt of the protocol.
- 2) If successfully emitted, photons are transmitted via optical channels in which photons travel at the speed of light. The speed of light in optical fiber is 2×10^8 m/s. Assuming that the length of the optical fiber from one QPU to the central BSM station is roughly 10m in a DQC center, the on-way transmission time is then ~ 50 ns.
- 3) Latency is introduced when extracting, processing, and transmitting classical measurement results from detectors. Even after obtaining and processing the measurement outcomes, latency

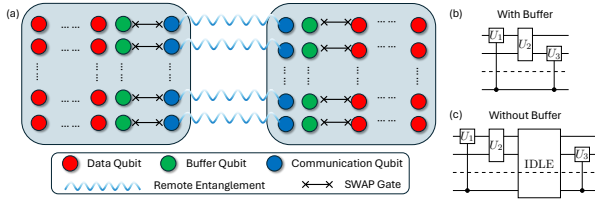


Fig. 2. Schematics of the DQC hardware architecture. (a) Example two-node scenario. (b) Example circuit segment which includes remote gates with buffer qubits. (c) Without a buffer qubit there is additional idling before a remote gate can be implemented due to the wait for entanglement generation.

may be introduced when transmitting the classical feedback using classical communication protocols. The lower bound for this last step is the physical transmission of information at the speed of light.

As a result, the total duration (cycle time) T_{EG} of a single heralded remote entanglement generation attempt is much longer than the cycle times for local operations T_{local} . For the typical case we assume $T_{EG} \geq 10T_{local}$ [15].

B. Entanglement buffering

We advocate the use of three types of qubits: buffer qubits, data qubits used to evaluate quantum circuits, and communication qubits which generate entanglement. A schematics of the architecture involving the three different types of qubits is shown in Fig. 2(a). After a pair of communication qubits successfully generates entanglement, each host node applies a local SWAP gate between its communication qubit and a buffer qubit. This results in storing the entangled state in the buffer qubit, whereas the communication qubit is freed up and can continue to be used in entanglement generation attempts. The buffer qubits can store multiple entanglement links to fulfill the demand for remote gates when needed. Decoupling EPR pair generation from storage also enables the pre-initialization of EPR pairs prior to program execution. As demonstrated in Section V, pre-initialized EPR pairs in the buffer (the `init_buf` design) reduce overall circuit latency. We note that QuComm [15] also formalized the concept of buffering in DQC to ensure sufficient communication resources for collective communication. In contrast, our work emphasizes the critical role of buffering in addressing the challenges of probabilistic EPR pair generation and evaluates the effectiveness of buffer qubits in this context. Next we provide additional motivation for introducing buffer qubits with focus on the system design perspective.

Layering: The introduction of buffer qubits in addition to communication qubits and data qubits allows separation of the entanglement generation process in DQC. With buffer qubits, entanglement generation can be implemented as a service that runs continuously [34]–[36] in the background. This hierarchical structure is beneficial for modularization of the DQC hardware architecture, thus simplifying the maintenance of each module. On the other hand, the interaction between communication qubits and buffer qubits to implement the SWAP gate can be realized with quantum interconnect technologies [37], [38].

Multiplexing: By attempting entanglement generation in parallel (multiplexed) using multiple available communication qubits, the latency for receiving an EPR pair is reduced due to the increased effective entanglement generation rate. However, it is unlikely that the entanglement links are generated exactly at the time when the remote gates request them, so they must be stored. If there are no buffer qubits, the communication qubits have to also serve as quantum

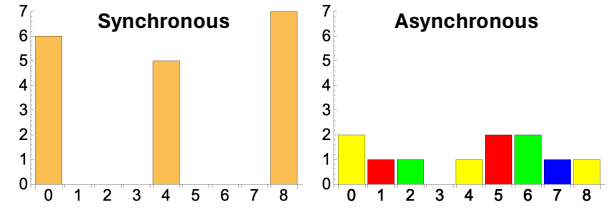


Fig. 3. Example entanglement generation patterns in the time domain. Synchronous and asynchronous attempts between each communication qubit are depicted. The vertical axis denotes the number of entangled pairs generated in one time unit, and the horizontal axis denotes time.

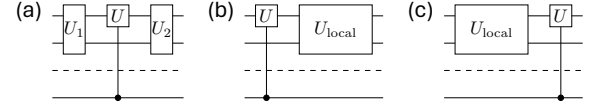


Fig. 4. Example circuit segment variants for different scheduling strategies. (a) Original circuit that does not consider the entanglement generation pattern. (b) ASAP implementation of the remote gate. (c) ALAP implementation of the remote gate.

memories. This means that fewer communication qubits are available and the effective entanglement generation rate decreases. Moreover, an insufficient number of available communication qubits can lead to unnecessary idling due to having no available entanglement links, leading to qubit state degradation and longer runtimes. Such a scenario is illustrated in Fig. 2(b) and (c).

C. Asynchronous remote entanglement generation

The cycle time for remote entanglement generation is generally much longer than the time scale of local quantum operations. For this reason we propose the use of asynchronous entanglement generation among all available communication qubits, a process that is able to “smoothen” the entanglement generation temporal pattern. This yields better resource allocation in the time domain.

In Fig. 3, we visualize the number of successfully generated entanglement links over time, in units of local operation cycle time T_{local} , and without loss of generality we have assumed $T_{EG} = 4T_{local}$. In the left panel, when all entanglement generation attempts are synchronous, the entanglement links will arrive in bursts. Remote gates thus need to wait for the burst arrival of entanglement links. After some links in the burst are consumed, the remaining links have to be stored in buffer qubits, which leads to idling decoherence, thus lowering the fidelity of remote gates. In contrast, the right panel demonstrates the pattern of asynchronous attempt when communication qubits are divided into 4 sub-groups, whose starting times of entanglement generation attempts are separated by T_{local} . Different colors denote different sub-groups. In this way, the number of entanglement generated within one time unit is lower, but they distribute more uniformly over time so that some remote gates are able to utilize the entanglement links as soon as they are generated. Additionally, the burst arrival of entangled states can lead to excessive waste when we apply a cut-off policy to buffer qubits, i.e. reset buffer qubits if they store entanglement for too long to avoid too much decoherence in entangled states.

D. Adaptive scheduling of remote gates

Adaptive scheduling of remote gates represents an additional optimization opportunity that can be implemented in software. Note

that for simplicity this work assumes all remote operations are implemented through gate teleportation, and we leave the simultaneous inclusion of both state and gate teleportations as a future work.

Information about existing generated entanglement links can be exploited by the DQC controller. Before a remote gate is executed, we examine whether there are already enough entanglement links stored in the buffer qubits. If yes, we can execute the remote gate earlier so that it can consume existing entanglement as-soon-as-possible (ASAP). This will create a longer time interval between the executed remote gate and subsequent remote gates, and the communication qubits will have a higher chance to create additional entanglement links demanded by the subsequent gates. If there are not enough entanglement links available yet, we can push the remote gate back (as-late-as-possible, ALAP) so that there is more time to generate the required entanglement.

This adaptive circuit modification process must be performed in real time. Because dynamically resynthesizing the quantum circuit based on the number of available EPR pairs e in the buffer is difficult, we can instead statically pre-compile multiple versions of the circuit. To ensure the scalability of the classical compilation, instead of recompiling the whole circuit, we partition the circuit into segments and use a look-up table strategy to manage them. Each segment is compiled with different scheduling policies, and the appropriate circuit version is selected in real time based on the number of available EPR pairs e . To achieve this, we first identify the remote gates and partition the original circuit into segments, where each segment contains m remote gates. The parameter m is tunable and is set to the product of the number of communication qubits and the EPR pair generation rate p_{succ} in our experiments. After partitioning the circuit, we can obtain the variants of the circuit segment corresponding to ASAP and ALAP, respectively, as illustrated by the examples in Fig. 4. During execution, if $e > m$, the controller looks up the next segment with ASAP policy. If $e = 0$, the controller opts for the ALAP policy. Otherwise, the controller uses the original scheduling. We leave more complicated scheduling strategies for future work.

IV. EVALUATION METHODOLOGY

In this section we describe our performance evaluation benchmarks, system configurations, comparison baselines, and figures of merit. We also describe the entanglement generation dynamics simulation.

A. Benchmarks and Configuration

Benchmarks: We evaluate 6 benchmarks that span different problem sizes and applications. The benchmarks are selected based on the proportion of remote gates in their circuits. The 1D Transverse-Longitudinal Ising Model (TLIM) circuit [39] features linear connectivity and a small number of remote gates. The Quantum Approximate Optimization Algorithm (QAOA) [40] is used to solve the MaxCut problem on regular graphs of degrees 4 and 8, and these circuits therefore have a medium proportion of remote gates. For instance, QAOA-r4-32 represents a 32-qubit QAOA circuit designed to solve the MaxCut problem on a degree-4 regular graph. Finally, the Quantum Fourier Transform (QFT) [41] benchmark requires full connectivity and exhibits a high proportion of remote gates. The benchmark properties are listed in Table I. The code and data that support the findings of this work are available upon request from the authors.

System Configuration: The properties of the different types of quantum operations are listed in Table II. The operation latencies

TABLE I
BENCHMARK PROPERTIES

Name	#qubits	#local 2Q ^a	#remote 2Q ^a	#1Q ^a	depth
TLIM-32	32	300	10	640	40
QAOA-r4-32	32	52	12	64	21
QAOA-r8-32	32	91	34	64	64
QFT-32	32	240	256	32	63
QAOA-r4-64	64	104	28	128	24
QAOA-r8-64	64	174	82	128	84

^a #local 2Q denotes the number of local two-qubit operations, #remote 2Q denotes the number of remote two-qubit operations, and #1Q denotes the number of single-qubit operations

listed in the table agree with those in [15]. In our evaluations, we assume that the success probability of one round of entanglement generation is $p_{\text{succ}} = 0.4$, the backend has a decoherence time $1/\kappa = 150\mu\text{s}$ where κ is the decoherence rate, and the local CNOT gate time is 300ns .

TABLE II
QUANTUM OPERATION PROPERTIES

Name	Latency	Fidelity
1Q gates	0.1	99.99%
Local CNOT gates	1	99.9%
Measurement	5	99.8%
EPR pair preparation	10	99%

Baseline: Similar to prior work [42], we utilize the METIS partitioning solver [43] to determine the partitions that are to be assigned to different nodes, aiming to minimize the number of remote operations.

B. Figures of merit

We evaluate different designs by comparing the circuit depth and the circuit fidelity. The circuit depth represents the total latency of the circuit. A depth of one corresponds to the latency of a single local CNOT gate. The circuit fidelity represents the quality of the final output. The fidelity is estimated as the product of fidelities of all local single-qubit gates, local two-qubit gates, remote gates implemented through gate teleportation, and an additional idling decoherence factor which accounts for latency. The local gate fidelities we use are listed in Table II. It is important to note that the fidelity of remote gates depends on the fidelity of the consumed entanglement link, which is affected by the buffer qubit decoherence. We use the phenomenological exponential decay factor $\exp(-\kappa t)$, where t is the idling time, to model idling decoherence.

C. Simulation of remote entanglement generation

Inspired by a recent open-source ad hoc quantum network simulation [44], we simulate asynchronous entanglement generation by communication qubits and storage of entanglement by buffer qubits. The system parameters are documented in Sec. IV-A. We assume that the initially generated Bell state is in the Werner form, i.e., a mixture of a pure Bell state and a 2-qubit maximally mixed state. We also assume that all buffer qubits have an identical decoherence rate, and the decoherence channel is the unbiased depolarizing channel, i.e. the three Pauli errors have an identical probability. As a result, the idling dynamics of the Bell state fidelity is given by $F(t) = F_0 \exp(-2\kappa t) + [1 - \exp(-2\kappa t)]/4$, where F_0 is the initial fidelity. The fidelity of a remote gate is obtained through the evaluation of the gate teleportation circuit which includes a noisy Bell state, noisy local 2-qubit gates, and a noisy single-qubit measurement.

V. RESULTS

In this section, we evaluate the DQC circuit depth and fidelity across various architecture designs and system sizes. The designs we consider include *original*: without any buffer qubits, *sync_buf*: with buffer and synchronous EPR pair generation, *async_buf*: with buffer and asynchronous EPR pair generation, *adapt_buf*: with asynchronous EPR pair generation and adaptive remote gate scheduling, *init_buf*: with pre-initiated EPR pairs in buffers, and *ideal*: execution on a monolithic device without remote operations. All reported results represent the average of 50 runs.

A. Comparison of different designs

We evaluated different designs on a simulated 2-node 32-data-qubit DQC architecture with 16 fully-connected data qubits assigned to each node. Each node contains 10 additional communication qubits and 10 buffer qubits. The results are presented in Figure 5 and 6. In Figure 5, we compare the circuit depth across different designs. The y-axis represents the circuit depth relative to the ideal circuit depth. The *original* design, which lacks buffer qubits, results in significant EPR pair waste and a substantial increase of the circuit depth. As shown in the figure, the largest reduction of the depth is achieved by leveraging buffer qubits. The *sync_buf* design reduces the circuit depth by 61.7%. However, the synchronous EPR pair generation pattern in *sync_buf* does not align well with the remote gate request pattern. The asynchronous EPR pair generation design *async_buf* yields an additional average 7% reduction of the circuit depth. Building on this, the adaptive scheduling *adapt_buf* further reduces the depth. By combining the pre-initialized EPR pairs and the adaptive scheduling, *init_buf* achieves an additional 7.5% depth reduction compared to the non-adaptive *async_buf* design.

We also compared the estimated circuit fidelity across different designs. As shown in Figure 6, the *async_buf* and *adapt_buf* designs achieve the same fidelity, yielding an average improvement of $2\times$ and $1.32\times$ compared to the *original* and *sync_buf* designs, respectively. Although the *init_buf* design leverages the pre-initiated EPR pairs to shorten the circuit depth, the extended idle time of these EPR pairs can lead to reduced fidelity.

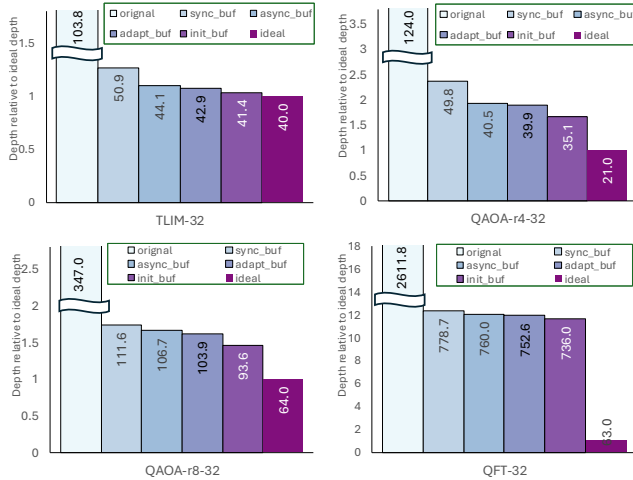


Fig. 5. Comparison of circuit depths across benchmarks and designs.

B. Impact of the number of communication qubits

We evaluate performance with a different number of communication and buffer qubits. The results are shown in Figure 7. We pick the

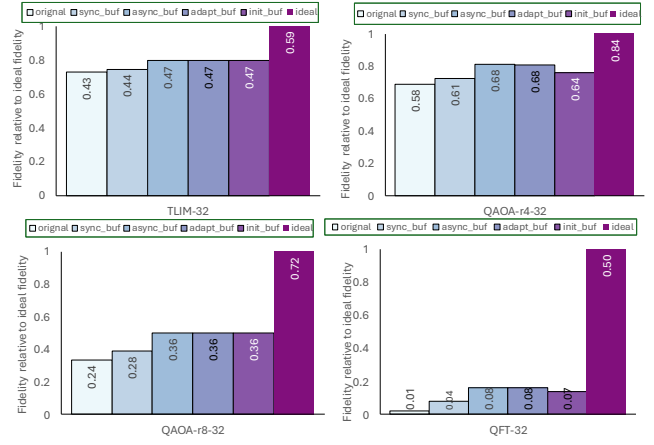


Fig. 6. Comparison of circuit fidelities across benchmarks and designs.

QAOA-r8-32 benchmark since the TLIM benchmark already achieves a near-ideal depth with 10 communication qubits, whereas the QFT-32 benchmark involves an excessive number of remote gates. The *original* design is excluded as it shows minimal change, allowing us to focus on the optimized designs. This also emphasizes that merely increasing the number of communication qubits is ineffective, as EPR pairs are wasted without proper storage in the buffer. As we increase the number of communication and buffer qubits, we notice a significant reduction of the circuit depth. Among the designs, *init_buf* consistently delivers the best performance. When the number of communication qubits reaches 20, it achieves a near-ideal depth, indicating that all remote gates are served immediately. Despite the significant reduction of the circuit depth, the circuit fidelity remains almost unchanged. Hence, we opted to not include the figure. The primary reason is that with asynchronous and adaptive scheduling EPR pairs are consumed immediately after generation, maintaining high fidelity across different cases. The relatively short circuit depth also minimizes the impact of decoherence errors, resulting in negligible differences of the fidelity.

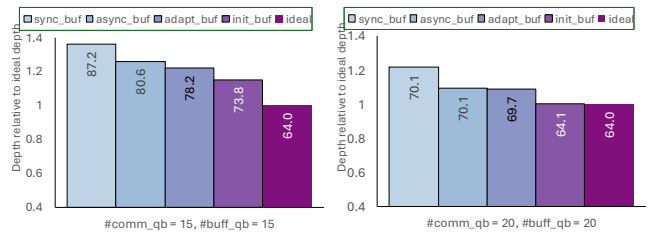


Fig. 7. Circuit depth of QAOA-r8-32 with different numbers of communication and buffer qubits.

C. Evaluations of larger systems

We evaluate two QAOA benchmarks on a 2-node 64-data-qubit system with 32 data qubits allocated to each node. Each node contains 20 additional communication qubits and buffer qubits. The circuit depth comparison is presented in Figure 8. The *init_buf* design achieves a 12% circuit depth reduction compared to the *sync_buf* design. As illustrated, our proposed designs continue to significantly reduce the circuit depth for larger system sizes.

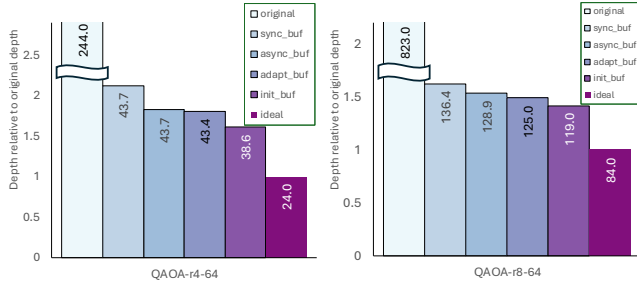


Fig. 8. Comparison of circuit depths across benchmarks and designs on a 64-qubit system.

VI. CONCLUSION AND DISCUSSION

In this paper, we study a new DQC architecture that leverages hardware and software co-design. One of our key observations is that buffering of successfully generated entanglement significantly reduces the circuit latency. Additionally, we introduce an asynchronous entanglement generation strategy that optimizes resource allocation and an adaptive scheduling mechanism that dynamically responds to real-time EPR pair availability. These advancements collectively enhance the performance and scalability of DQC systems, paving the way for more practical and efficient implementations that support the needs of real-world applications.

ACKNOWLEDGMENTS

This material is based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers. This material is also based upon work supported by the DOE-SC Office of Advanced Scientific Computing Research MACH-Q project under contract number DE-AC02-06CH11357. A.Z. and T.Z. acknowledge support from the NSF Quantum Leap Challenge Institute for Hybrid Quantum Architectures and Networks (NSF Award 2016136).

REFERENCES

- [1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, 1999.
- [2] D. Cuomo *et al.*, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, 2020.
- [3] M. Caleffi *et al.*, "Distributed quantum computing: a survey," *Computer Networks*, vol. 254, 2024.
- [4] D. Barral *et al.*, "Review of distributed quantum computing. From single GPU to high performance quantum computing," *arXiv:2404.01265*, 2024.
- [5] A. Carrera Vazquez *et al.*, "Combining quantum processors with real-time classical communication," *Nature*, pp. 1–5, 2024.
- [6] F. Afzal *et al.*, "Distributed quantum computing in silicon," *arXiv:2406.01704*, 2024.
- [7] D. L. Moehring *et al.*, "Entanglement of single-atom quantum bits at a distance," *Nature*, vol. 449, no. 7158, 2007.
- [8] S. Ritter *et al.*, "An elementary quantum network of single atoms in optical cavities," *Nature*, vol. 484, no. 7393, 2012.
- [9] J. Hofmann *et al.*, "Heralded entanglement between widely separated atoms," *Science*, vol. 337, no. 6090, 2012.
- [10] H. Bernien *et al.*, "Heralded entanglement between solid-state qubits separated by three metres," *Nature*, vol. 497, no. 7447, 2013.
- [11] J.-W. Pan *et al.*, "Experimental entanglement swapping: entangling photons that never interacted," *Phys. Rev. Lett.*, vol. 80, no. 18, 1998.
- [12] C. H. Bennett *et al.*, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Phys. Rev. Lett.*, vol. 70, no. 13, 1993.

- [13] D. Gottesman *et al.*, "Demonstrating the viability of universal quantum computation using teleportation and single-qubit operations," *Nature*, vol. 402, no. 6760, 1999.
- [14] A. Wu *et al.*, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *IEEE/ACM MICRO*, 2022.
- [15] —, "QuComm: Optimizing collective communication for distributed quantum computing," in *IEEE/ACM MICRO*, 2023.
- [16] M. Zomorodi-Moghadam *et al.*, "Optimizing teleportation cost in distributed quantum circuits," *Int. J. Theor. Phys.*, vol. 57, 2018.
- [17] P. Andres-Martinez *et al.*, "Automated distribution of quantum circuits via hypergraph partitioning," *Phys. Rev. A*, vol. 100, no. 3, 2019.
- [18] Z. Davarzani *et al.*, "A dynamic programming approach for distributing quantum circuits by bipartite graphs," *Quantum Inf. Process.*, vol. 19, 2020.
- [19] O. Daei *et al.*, "Optimized quantum circuit partitioning," *Int. J. Theor. Phys.*, vol. 59, no. 12, 2020.
- [20] D. Ferrari *et al.*, "Compiler design for distributed quantum computing," *IEEE TQE*, vol. 2, 2021.
- [21] S. DiAdamo *et al.*, "Distributed quantum computing and network control for accelerated VQE," *IEEE TQE*, vol. 2, 2021.
- [22] D. Dadkhah *et al.*, "Reordering and partitioning of distributed quantum circuits," *IEEE Access*, vol. 10, 2022.
- [23] D. Ferrari *et al.*, "A modular quantum compilation framework for distributed quantum computing," *IEEE TQE*, 2023.
- [24] J. M. Baker *et al.*, "Time-sliced quantum circuit partitioning for modular architectures," in *ACM CF*, 2020.
- [25] M. Bandic *et al.*, "Mapping quantum circuits to modular architectures with QUBO," in *IEEE QCE*, 2023.
- [26] P. Escofet *et al.*, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM TQC*, 2024.
- [27] P. Promponas *et al.*, "Compiler for distributed quantum computing: a reinforcement learning approach," *arXiv:2404.17077*, 2024.
- [28] E. Russo *et al.*, "Attention-based deep reinforcement learning for qubit allocation in modular quantum architectures," *arXiv:2406.11452*, 2024.
- [29] J. Ang *et al.*, "ARQUIN: Architectures for multinode superconducting quantum computers," *ACM TQC*, vol. 5, no. 3, 2024.
- [30] J. Kim *et al.*, "A fault-tolerant million qubit-scale distributed quantum computer," in *ACM ASPLOS*, 2024.
- [31] C. Chu *et al.*, "TITAN: A fast and distributed large-scale trapped-ion NISQ computer," in *ACM/IEEE DAC*, 2024.
- [32] S. Bahrani *et al.*, "Resource management and circuit scheduling for distributed quantum computing interconnect networks," *arXiv:2409.12675*, 2024.
- [33] N. Lütkenhaus *et al.*, "Bell measurements for teleportation," *Phys. Rev. A*, vol. 59, no. 5, 1999.
- [34] A. Kolar *et al.*, "Adaptive, continuous entanglement generation for quantum networks," in *IEEE INFOCOM Workshops*, 2022.
- [35] A. Zang *et al.*, "Analytical performance estimations for quantum repeater network scenarios," in *IEEE QCE*, 2024.
- [36] C. Zhan *et al.*, "Design and simulation of the adaptive continuous entanglement generation protocol," *arXiv:2502.01964*, 2025.
- [37] D. Awschalom *et al.*, "Development of quantum interconnects (QuICs) for next-generation information technologies," *PRX Quantum*, vol. 2, no. 1, 2021.
- [38] —, "A roadmap for quantum interconnects," Argonne National Laboratory, Tech. Rep., 2022.
- [39] A. Sopena *et al.*, "Simulating quench dynamics on a digital quantum computer with data-driven error mitigation," *Quantum Science and Technology*, vol. 6, no. 4, 2021.
- [40] E. Farhi *et al.*, "A quantum approximate optimization algorithm," *arXiv:1411.4028*, 2014.
- [41] D. Coppersmith, "An approximate Fourier transform useful in quantum factoring," *quant-ph/0201067*, 2002.
- [42] M. G. Davis *et al.*, "Towards distributed quantum computing by qubit and gate graph partitioning techniques," in *IEEE QCE*, 2023.
- [43] G. Karypis *et al.*, "METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices," 1997.
- [44] A. Zang *et al.*, "Entanglement distribution in quantum repeater with purification and optimized buffer time," in *IEEE INFOCOM Workshops*, 2023.